# Leveraging Sparsity in DNN Vector Algebra

TBD

March 12, 2026

**Abstract**

Your abstract.

## 1 Introduction

Artificial Intelligence (AI) and in particular Deep Neural Networks (DNN) have achieved better than human accuracy on many cognitive tasks involving images, natural language processing, and protein structure among others [1, 2, 3]. Deep learning consists of two phases: training and inference. At training phase, a deep learning model is developed by feeding the model with a large dataset. In most commonly used DNNs, learning from the input data is achieved through interleaving feed-forward and back-propagation paths. The machine predicts the meaning of the input at the feed-forward path while executing the computations from the first to the last layer. Once the last layer is reached, the back-propagation path is triggered, where the prediction and actual result are checked and the weight values of all layers are refined with respect to the prediction error while going backwards from the last layer to the first layer. After a neural network is trained, it is deployed to run inference to classify, recognize, and process new inputs.

The algorithmic superiority of Deep Learning comes at a high computational cost in terms of memory, execution time, and electric power. Most DNN training is done using massive parallel architectures such as computer clusters and GPUs. Once the DNN is trained, inference is not as demanding and trained DNN are usually deployed as static models. However, running DNN inference on mobile or small embedded devices, which are power and energy constrained can still be challenging. For resource constraint environments, Fixed-Point(FxP) quantization implementation has been a solution to alleviate the compute, memory and network demand [4, 5, 6]. Thus, quantization is not a new solution to reducing computing requirements in DNNs. However, often the choice of bit-width is constrained by the ability of fixed-point implementations to represent the entire range of values with sufficient resolution. The fixed low bit-width DNNs may suffer losses in classification accuracy due to quantization errors. Similarly, for DNN training, the training dataset sizes grow faster than advancements in DNN hardware [7]. A software solution to this problem, could allow for flexible bit-width selection, that can be combined with data compression. It could be deployed on processing units from different vendors with various architectures while increasing energy efficiency and processing speeds.

In this work we investigate bit-slicing vectors (BSV) as a potential data structure for storing DNN vectors for better control over quantization and better sparsity leverage. Due to the properties of the bit-slicing vectors, it is possible to choose the precision for each vector operation, by selecting the number of slices(bits) to be processed. This choice can be any number and is not limited to a few options such as, for example, not limited to powers of two. We have previously implemented dynamic quantization based on the query parameters in similarity queries [8].

The operations that carry the bulk of the processing load in DNN training and inference are vector primitive operations such as dot-product, summation, aggregation/pooling, and threshold functions [9]. In this work we describe algorithms that implement these operations with BSV data structures and evaluate their efficiency for potential use in machine learning systems.

Summary of results here.

# 2 Background and Related Work

In this section we will review the effect DNN training and inference has on the system. In the training phase, we have a forward and backward pass where the gradients and weights get updated. This typically requires a higher precision to maintain numerical stability for accurate gradient calculations that would help the model converge to an optimal solution. In the inference phase, we use lower precision numbers as compared to training as we tend to have computationally restrained deployment environments. A numerical accuracy loss is acceptable as long as the predictive performance is within an acceptable range. Overall, DNN training and inference consumes a significant amount of computing resources including CPUs, GPUs, memory and storage. Different techniques have been introduced over the years to optimize the training and inference of DNN's. In this section, we will broadly classify the optimizations into two categories namely hardware and software optimizations.

## 2.1 Hardware optimizations

Standard deep learning applications use 32-bits of floating point precision during training. Optimization is typically done during inference, however, research has shown that lower numerical precision using 16-bit and 8-bit multipliers can still achieve similar accuracy levels while using lesser memory and performing higher number of operations per second [10]. As per [11] 16-bit fixed point numbers can be used to train state-of-the-art neural networks by employing stochastic rounding during fixed-point computations. [12] further reduced the number of bits required for training by introducing a new 8-bit floating point format (F P8) using chunk-based computations and stochastic rounding that achieved similar accuracy to full-precision training. It is even more challenging to create optimized mobile deep learning applications because of the limited computations resources available compared to desktop or server machines. [13] proposed a unified neural processing unit for mobile DNN applications that allows for fully variable precision from 1 to 16 bit with the help of lookup table-based bit-serial processing element.

## 2.2 Software optimizations

When changes are made to some aspect of the software system such that it results in improved compute performance we call them software optimizations. We further breakdown software optimizations into DNN compression and model optimization.

### 2.2.1 DNN compression

DNN compression focuses on reducing the size and complexity of a trained neural network while retaining its predictive performance. The goal of compression is to make the model more lightweight and efficient for deployment and therefore reduce memory and computational requirements. Some of the techniques used to attain this include network pruning, sparse representation, bits precision, knowledge distillation [14]. We always try to have a smaller memory footprint. It is useful to do DNN compression when we are deploying models in real world applications.

Sparse-Quantized Representation (SpQR) [15] is a compressed format using quantization for the near-lossless (less than 1% error compared to dense baseline) compression of LLMs. It identifies and isolates outlier weights, which cause large quantization errors, and stores them in higher precision. To identify these sensitive weights, a small set of calibration inputs is run through the mode. The sensitivity is collected at each layer using an equation based on the difference between the original predictions and predictions based on quantized weights.

There are several patterns in sensitivity commonly found in outliers. Row outliers are regions of high sensitivity within an output unit. Column outliers are regions of high sensitivity in select input dimensions across all rows. Sensitive attention heads are regular stripes of width 128 corresponding to one attention head. The Rotary embedding pattern is a repeating vertical pattern of sensitivity where half the attention head is rotated with cosine, and the other half rotated with sine using same set of frequencies. Unstructured outliers are the weights that don't fit into above patterns, mostly in columns with the largest input index since that is where the most error is accumulated.

SpQR includes two changes to the quantization procedure. In capturing small groups of weights with bilevel quantization, weights are grouped into sets of 8 to 32 with their own quantization scale and

zero. Groupwise quantization statistics are then quantized using the same algorithm to further reduce overhead. Non-outlier weights are quantized into 3-4 bits while high sensitivity outliers are quantized into 16 bits. The sensitivity threshold should pick out about 1% of the weights to be outliers.

#### 2.2.2 Model Optimization

Model optimization refers to modifications in the model architecture to achieve lower memory consumption and reduced computational requirements while maintaining or improving model performance. A model optimization method proposed by [16] introduced quantization networks that used a new type of differentiable non linear activation function that can be applied to both weights and activations in deep neural networks. It uses a temperature factor T in the sigmoid function that controls the gap between the hard quantization function in the inference and soft quantization during training and finds the right balance between model learning capacity and quantization gap. The quantization network outperformed state-of-the-art methods on image classification and object detection. Another model optimization technique involves building an energy inference engine (EIE) [17] that prunes redundant model layers and have multiple connections share the same weight such that large DNNs can fit in on-chip SRAM. This engine results in energy saving, sparsity exploitation and faster results than the non-compressed DNN counterparts. One drawback of EIE is that it only exploited sparsity in the fully connected layers.

### 2.3 Software Hardware Optimization

Most techniques fail to find a balance between hardware acceleration and software compression. A hardware-software co-optimized reconfigurable framework for sparse CNNs is introduced by [18]. It makes use of a unique software-based load-balance aware pruning method and kernel-merging on FPGA's that effectively eliminate the useless MAC operations. This framework outperforms previous CNN FPGA accelerators when tested on AlexNet and VGG16. OMNI is another co-optimized framework that surpasses modern CNN processing time and energy efficiency by using software pruning techniques that balances the sparsity across different groups in the same pattern as the hardware memory partition [19].

[Our Work]

## 3 Methods

In this section, we will introduce our algorithms for implementing dot product, top K max, top K min, and sum on hybrid bit-sliced vectors and provide a summary of the results we achieved, alongside comparisons with existing technology.

### 3.1 Bit-Sliced Vectors

A vector is an array of numbers. The numbers are arranged in order and each individual number can be identified by its index in that ordering [20]. Bit-sliced indices(BSI) are used to encode attributes and perform queries over high dimensional data using bit-wise operations [21]. The absolute value of numbers is stored in a hybrid bitmap in either compressed or uncompressed form, indicated by run-length words, followed by literal or filler words that store the actual value. The sign of numbers is stored in a separate hybrid bitmap, 1 in the bitwise representation where there is a negative number and 0 where there is a positive number. The existence bitmap is used to track the existence of a number at a certain index or the lack thereof due to deletion. This paper adapts the idea of BSI to formulate a new way of representing vectors as Bit-sliced Vectors(BSV).

### 3.2 Top K Preference Queries

Top k preference queries describe any algorithms used to find the k elements that most closely fit a certain metric. In this section, we will introduce algorithms for implementing top k maximum and top k minimum using bit-sliced vector properties. Top $k$ maximum is an algorithm to find an unordered

| Vector | Bit Sliced Vector | | |
|---|---|---|---|
| | Slice 0 | Slice 1 | Slice 2 |
| 1 | 0 | 0 | 1 |
| 2 | 0 | 1 | 0 |
| 3 | 0 | 1 | 1 |
| 4 | 1 | 0 | 0 |

Table 1: An example of a Bit Sliced Vector

list of the $k$ greatest integers in a given vector of integers. Top $k$ minimum is an algorithm to find an unordered list of the $k$ smallest integers in a given vector of integers.

Consider top K maximum for positive numbers. When a number is the greatest in the vector, its binary representation will have the largest number of significant bits. For example, in the vector

$$[2, 3]$$

3 is the largest number, and in its binary representation 11 it has two significant bits. No other number has a larger number of significant bits, although 2, with the binary representation 10, may have an equal number of significant bits. Since bit-sliced vectors store separate slices for each order of significance (see details in [21]), the integers with the largest size in bits can be extracted using the bitwise AND operation between the slice at the most significant index in the existence bitmap and in the value bitmap. This operation will result in an existence bitmap of the positions of the largest $n$ numbers. If $n$ is larger than $k$, this existence bitmap will become the new existence bitmap we are working with since we do not need to know the existence of any smaller numbers that are not included in this bitmap anymore. If $n$ is less than $k$, then we will continue to operate on the original existence bitmap, using the bitwise OR operation to combine the current result with future results. Until $n$ reaches $k$, we will continue this operation for each order of significance, starting from the most significant digit.

**Algorithm 1:** Top K Max (Positive): Given an integer $k$ and a bit-sliced vector $BSV$, we return a bitmap representing the positions of the $k$ greatest positive integers.

```
Input: BSI, k
Output: topK
SIGN = BSI.signBitmap;
E = BSI.existenceBitmap;
BSV = BSI.bsv;
E = NOT(SIGN) AND E ;                    // E is replaced with the positions of positive
                                                                          // numbers only.
n = 0 for i = BSV.size − 1; i >= 0; i− ;                // While there are bit-slices in BSV.
do
    SE = E AND BSV[i] ;                          // Find the position of numbers with 1 in this slice.
    X = topK OR SE ;                        // Combine result from current slice with previous slices.
    n = NumberOfOnes(X) ;                          // Count how many numbers are found in the result.
    if n > k then
        E = SE ;                        // Replace the existence bitmap with the result from this slice.
    end
    else if n < k then
        topK = X
        E = E AND NOT(BSV[i]) ;     // Replace the existence bitmap with positions of numbers not
                                                                          // considered yet.
    end
    else
        E = SE
        break
    end
end
topK = topK OR E
```

Algorithm 1 shows the implementation of top K maximum on positive numbers only. However, in real neural network applications, numbers are given in both positive and negative forms. In both sign magnitude form and twos complement form, positive numbers and negative numbers are compared

separately, so we also have to create a separate algorithm for dealing with negative cases. Similar to top K maximum on positive numbers, top K maximum on negative numbers also starts out from the most significant slice. Taking the existence bitmap, we eliminate numbers with 1 in the slice because they have a greater value. If the resulting bitmap has less than $k$ numbers, we rollback our changes by performing the AND operation between the existence bitmap and the current bit slice in order to bring back the larger numbers that have 1 in the current bit slice. If the resulting bitmap has greater than $k$ numbers, we replace the existence bitmap with the new bitmap. Continuing until either the resulting bitmap reaches exactly $k$ numbers or reaches the end of the bit slices, we parse through each slice and perform these operations repeatedly.

**Algorithm 2:** Top K Max (Negative): Given an integer $k$ and a bit-sliced vector $BSV$, we return a bitmap representing the positions of the $k$ greatest negative integers.

```
Input: BSI, k
Output: topK
SIGN = BSI.signBitmap;
E = BSI.existenceBitmap;
BSV = BSI.bsv;
E = SIGN AND E ;                    // E is replaced with the positions of negative
                                                                        // numbers only.
n = 0 for i = BSV.size − 1; i >= 0; i− ;          // While there are bit-slices in BSV.
do
    SE = E AND NOT(BSV[i]) ;          // Find the position of numbers with 0 in this slice.
    X = topK OR SE n = NumberOfOnes(X) if n > k then
        E = SE
    end
    else if n < k then
        topK = X
        E = E AND BSV[i] ;      // Replace the existence bitmap with positions of numbers not
                                                                        // considered yet.
    end
    else
        E = SE
        break
    end
end
topK = topK OR E
```

The final top K max algorithm will call on top K max on positive numbers first and call on top K max on negative numbers with $n - k$ as $k$ if there are not enough positive numbers. Top K minimum uses the same algorithms as top k maximum with the only difference in the order that positive and negative numbers are processed in. Instead of operating first on positive numbers, top K min will use the first top K max algorithm on negative numbers, and vice versa.

## 3.3 Dot Product

The dot product operation is a mathematical operation that takes two vectors and produces a scalar value by adding the pairwise products of their corresponding elements. A widely applicable use for the dot product of two vectors in machine learning is the cosine similarity function. Bit-Sliced vectors(BSV), by encoding the individual elements of a vector using the binary representation of their values, give rise to an optimized way to calculate the dot product. Given two BSVs, the scaler dot product value can be calculated by first performing an AND operation on corresponding slices to identify the shared '1' bits. The count of '1' bits in the resulting slice is determined, and this count is then multiplied by a shifting factor and cumulatively added to derive the final dot product result. The sign bits (0 for positive and 1 for negative) are used to determine whether a particular element in the bit-sliced vectors should contribute positively or negatively to the dot product. The counts of ones for positive and negative contributions are accumulated in the res variable. Positive contributions are added, and negative contributions are subtracted. By leveraging bitwise operations and positional weighting, the algorithm offers an innovative approach for efficiently calculating dot products using BSVs.

**Algorithm 3:** Dot Product: Given two BSVs, A and B, we construct a scalar quantity, C = A.B, using the following pseudo-code.

```
Input: A, B
Output: C
C = 0;                                                    // Initialize the result
for i = 0; i < A.size; i++;                               // For each slice in A
do
    for j = 0; j < B.size; j++;                           // For each slice in B
    do
        R = R − A[i] AND B[j] AND (A[sign] XOR B[sign])
        R = R + A[I] AND B[j] AND NOT(A[sign] XOR B[sign]) ;   // Perform AND
         operation on the corresponding slices
        C = C + numberOfOnes(R) * (2^(j+i−1));   // Calculate the number of ones in
         the resulting BSV, shift and add to the result
    end
end
```

| Vector 1 | Vector 2 | Bit Sliced Vector 1 | | | Bit Sliced Vector 2 | | |
|---|---|---|---|---|---|---|---|
| | | Sign | Slice 1 | Slice 0 | Sign | Slice 1 | Slice 0 |
| 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 |
| 2 | -2 | 0 | 1 | 0 | 1 | 1 | 0 |
| 3 | -3 | 0 | 1 | 1 | 1 | 1 | 1 |

Table 2: An example of a Bit Sliced Vector Dot Product

# 4 Experimental Evaluation

## 4.1 Experimental Setup

The data sets for both signed and unsigned numbers were generated using the zipf distribution. For each operation, either the cardinality, size of vectors, or the zipf distribution value was varied while the others were kept constant. The compression threshold was held constant at 0.5. The results were averaged over 5 test runs and measured in microseconds. The code was written in C++ and run on 64-bit machines.

## 4.2 Top K Queries

## 4.3 Dot Product

We performed two sets of evaluations for the dot product operation between two one-dimensional vectors. In both assessments, we randomly generated the vector elements, adhering to a Zipf distribution, with roughly half of them being negative. The numbers were constrained to a maximum bit length of 16.

In the first evaluation, we maintained a constant vector length, setting it initially at 10,000 and later at 1,000,000 (1M). During this evaluation, we adjusted the Zipf parameter within the range of 0 to 3. It was observed that when using higher Zipf exponent values, the BSV dot product yielded results comparable to those obtained from the standard vector dot product. This observation holds promise, indicating that in real-world scenarios characterized by the presence of Zipf's law, the combination of hybrid compression and optimized bit slice vector operations has the potential to deliver high-performance results.

In the second set of assessments, we kept the Zipf exponent to a constant value of 1 and varied the length of the vector from 100 to 10000000(10M). The vector dot product demonstrated superior performance, while the BSV dot product consistently aligned, indicating a strong positive correlation with the standard vector dot product in a logarithmic scale. When the zipf exponent was changed to 3, the time taken on a logarithmic scale for the BSV dot product was comparable to that of the regular dot product.

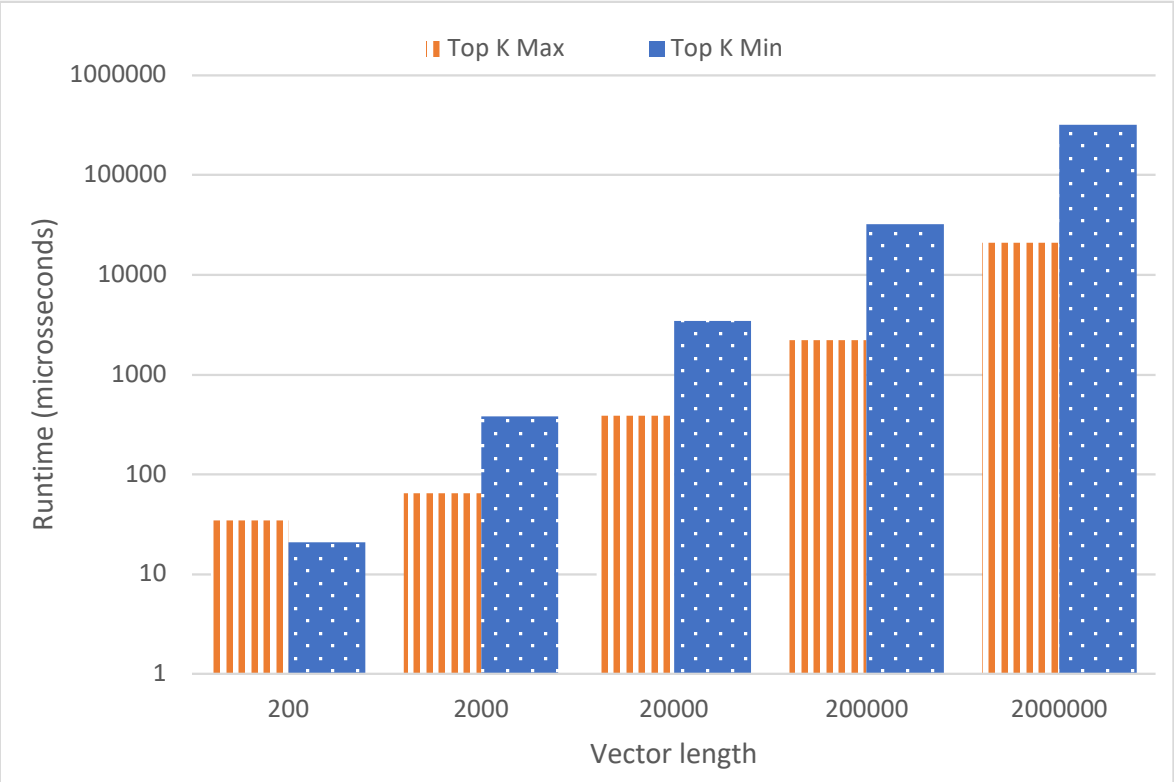


Figure 1: Average top K max runtime with k=5 and averaged over 5 runs compared to quicksort. The input is a vector with varying lengths of random integers ranging from $-2^{16}$ to $2^{16}$ generated by zipf algorithms on f = 1. The compression threshold for BSV is 0.5.

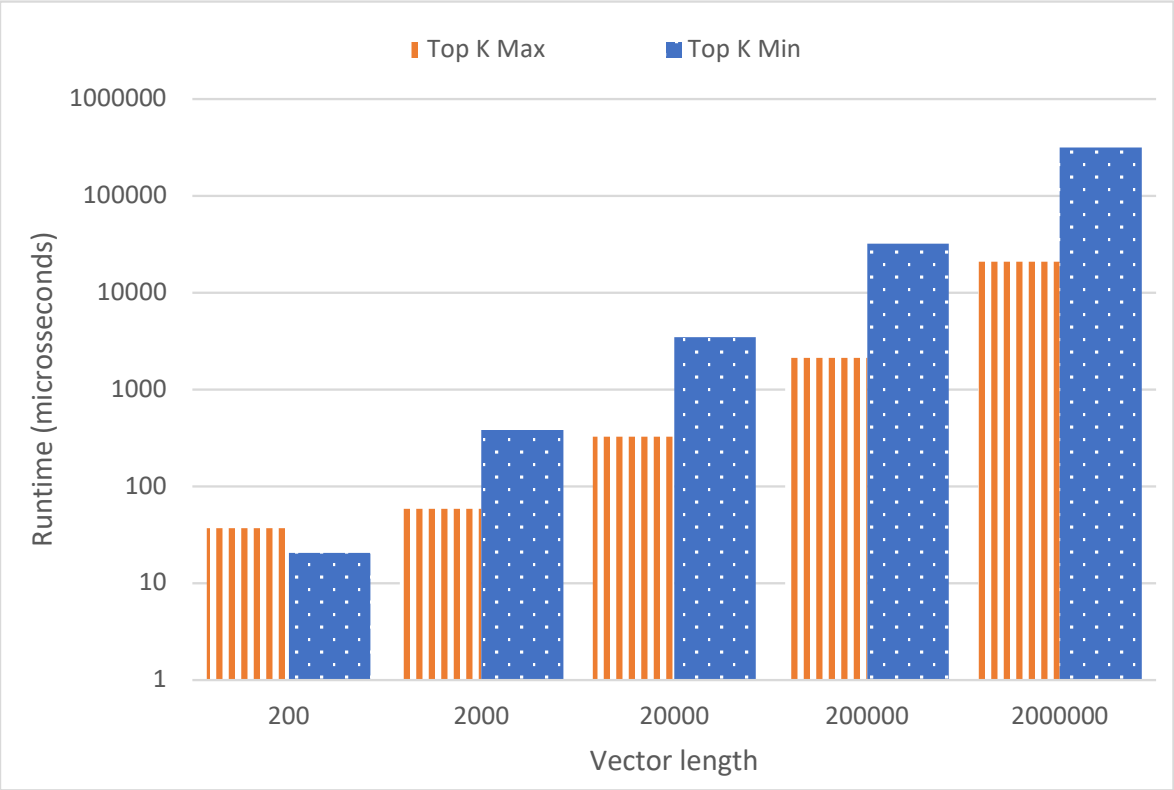


Figure 2: Average top K min runtime with k=5 and averaged over 5 runs compared to quicksort. The input is vector with varying lengths of random integers ranging from $-2^{16}$ to $2^{16}$ generated by zipf algorithms on f = 1. The compression threshold for BSV is 0.5.

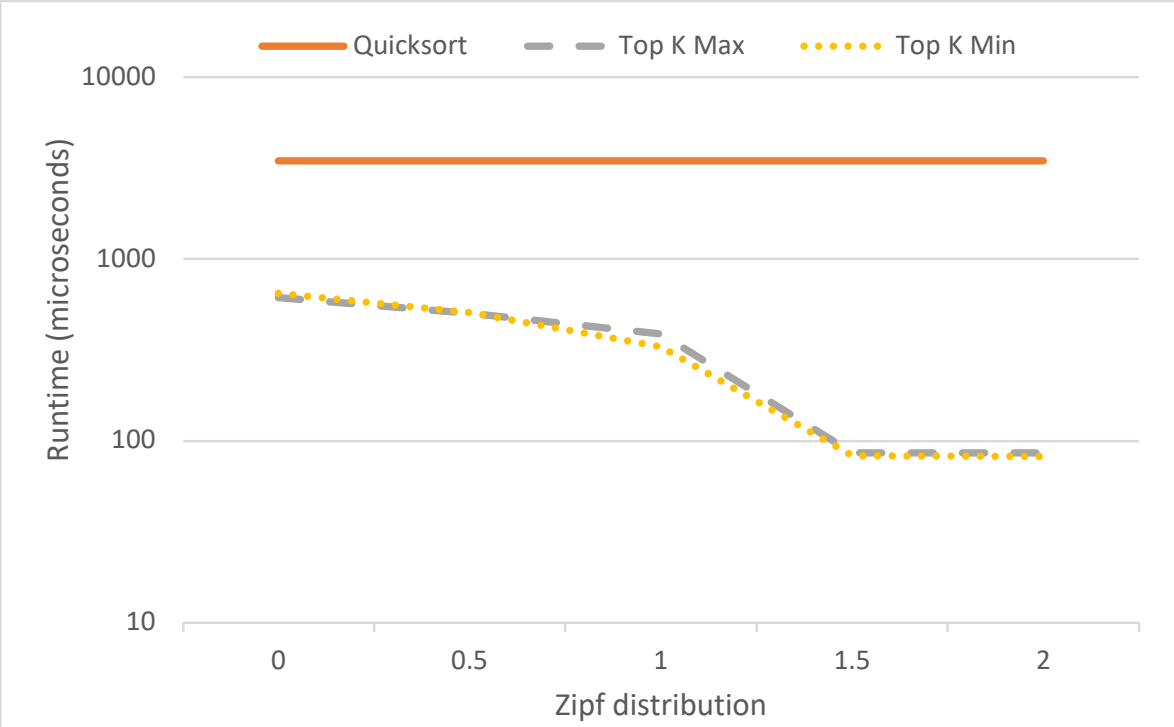


Figure 3: Average top K max and top K min runtime with k=5 and averaged over 5 runs compared to quicksort. The input is a vector of 20,000 random integers ranging from $-2^{16}$ to $2^{16}$ generated on a varied zipf distribution. The compression threshold for BSV is 0.5.

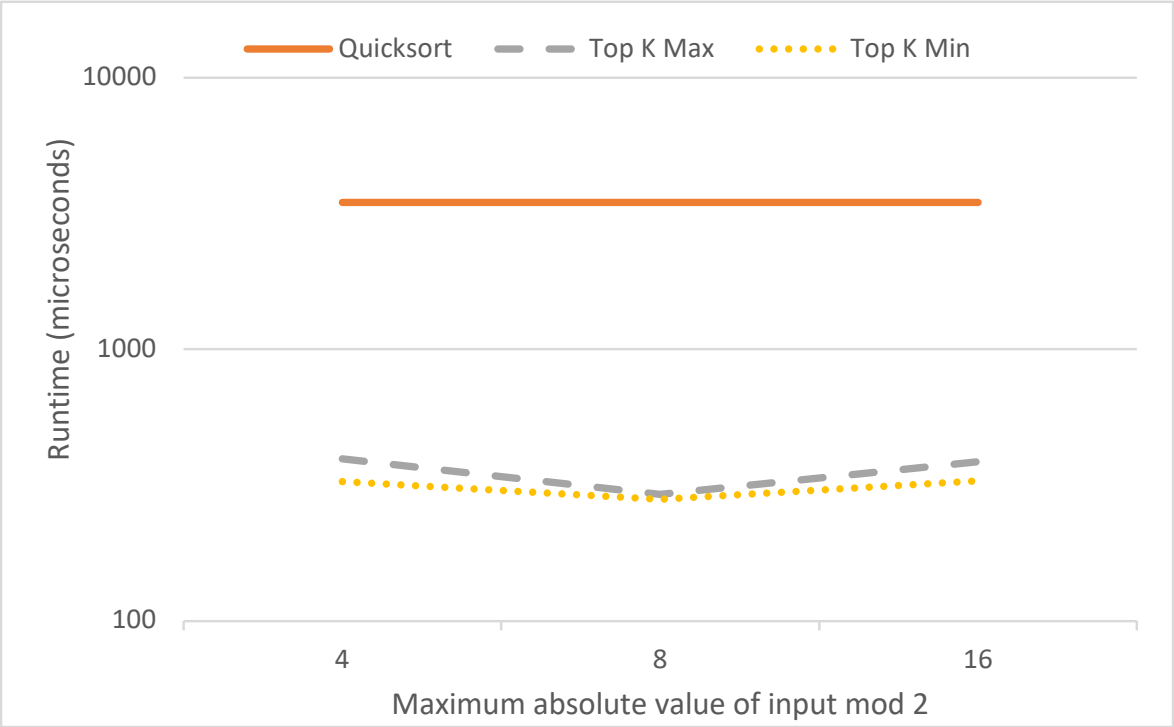


Figure 4: Average top K max and top K min runtime with k=5 and averaged over 5 runs compared to quicksort. The input is a vector of 20,000 random integers with varying range generated by zipf algorithms on f = 1. The compression threshold for BSV is 0.5.

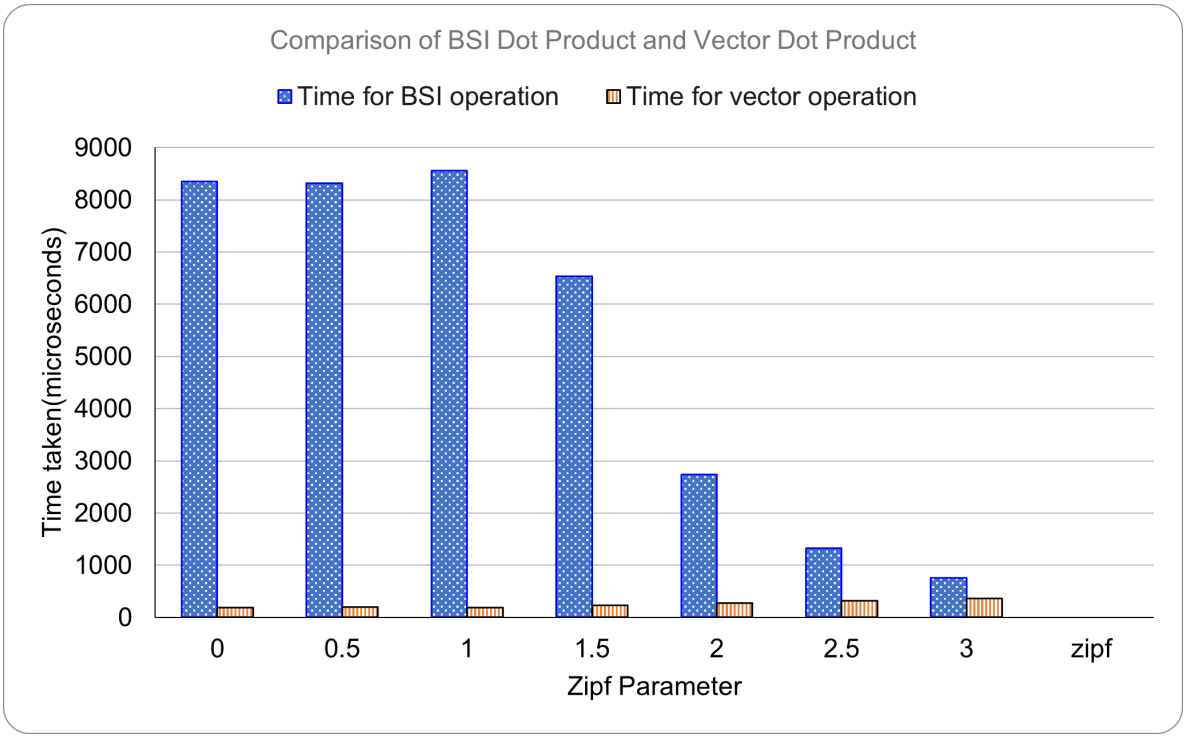


Figure 5: Dot Product runtime comparison for BSV and regular vector representations averaged over 5 runs. The input is a vector of 10,000 random integers with varying distributions generated by Zipf parameters ranging from 0 to 3. The compression threshold for BSV is 0.5 and the maximum number of bit slices is 16

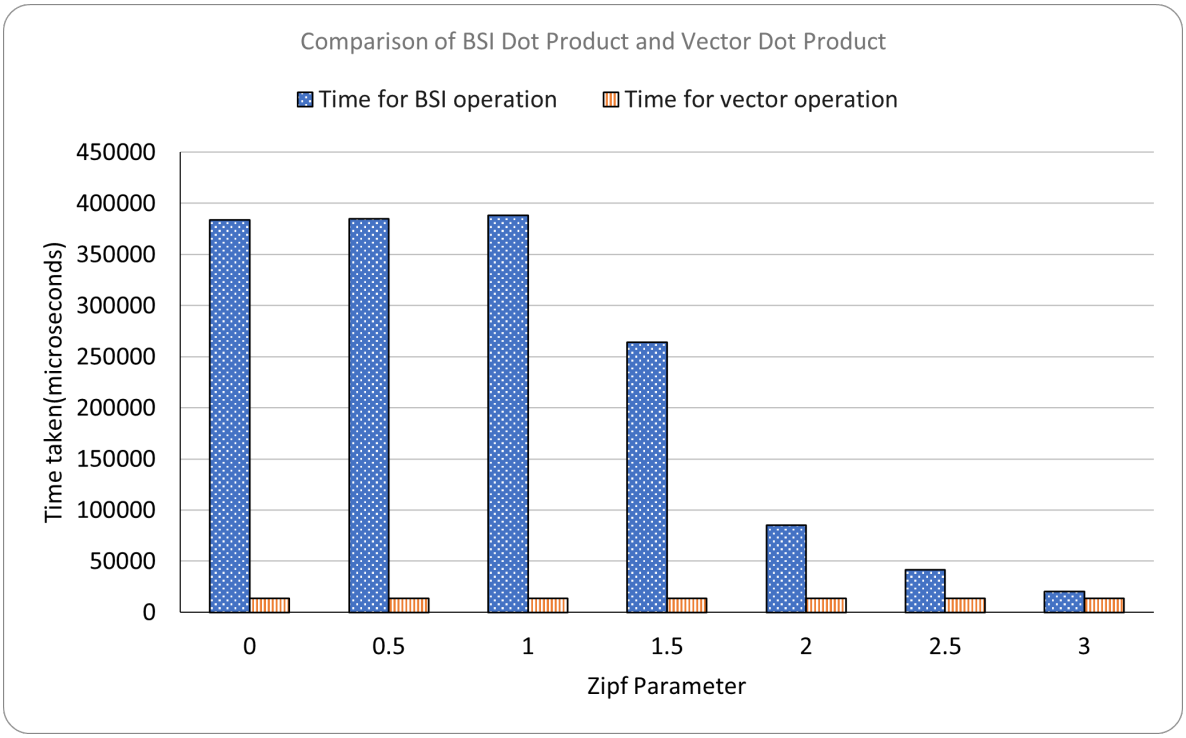


Figure 6: Dot Product runtime comparison for BSV and regular vector representations averaged over 5 runs. The input is a vector of 1M random integers with varying distributions generated by Zipf parameters ranging from 0 to 3. The compression threshold for BSV is 0.5 and the maximum number of bit slices is 16

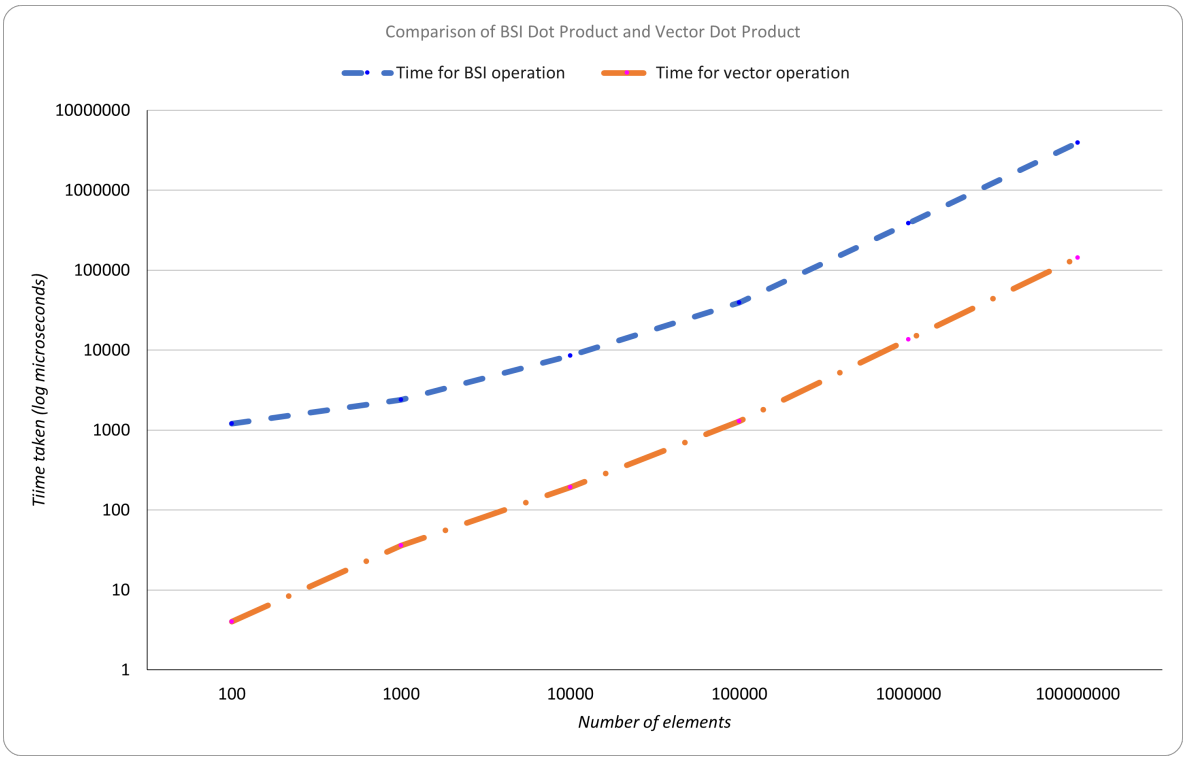


Figure 7: Dot Product runtime comparison for BSV and regular vector representations averaged over 5 runs. The input is a vector of random integers with varying lengths between 100 and 1M generated by Zipf parameter 1. The compression threshold for BSV is 0.5 and the maximum number of bit slices is 16

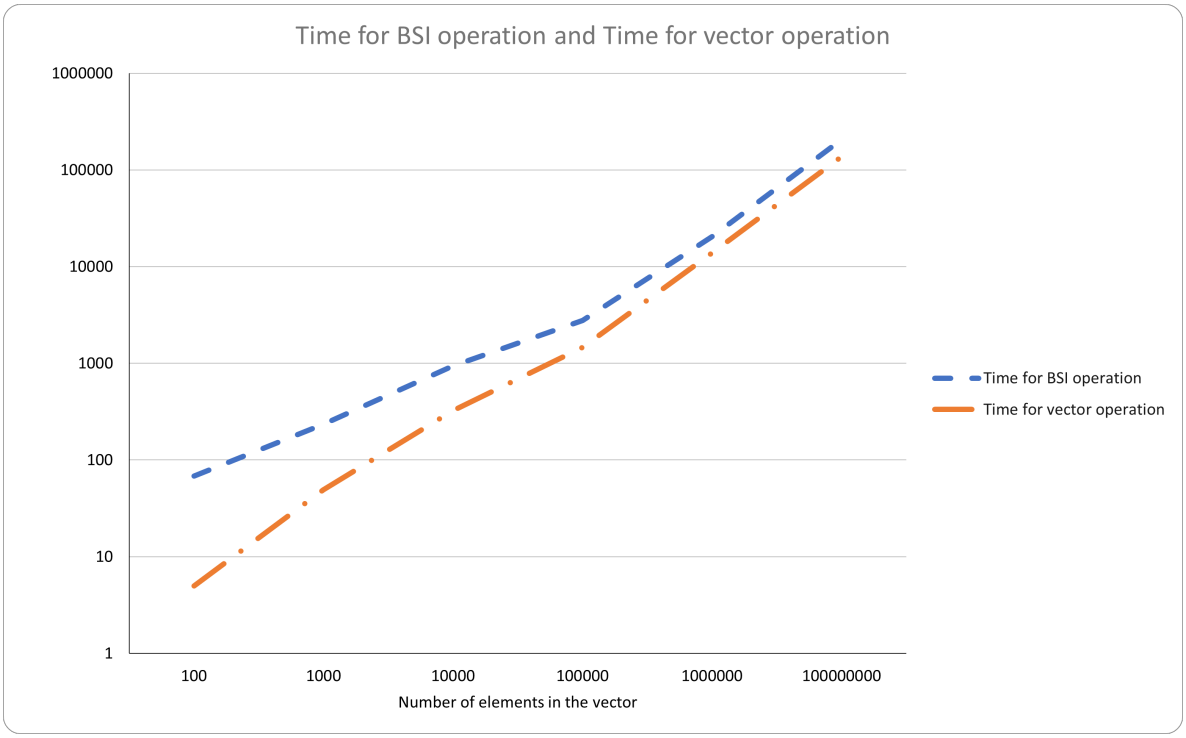


Figure 8: Dot Product runtime comparison for BSV and regular vector representations averaged over 5 runs. The input is a vector of random integers with varying lengths between 100 and 1M generated by Zipf parameter 3. The compression threshold for BSV is 0.5 and the maximum number of bit slices is 16

# 5 Conclusion

# References


[1] T. Maynard, L. Baldassarre, Y.-A. de Montjoye, L. McFall, and M. Óskarsdóttir, "Ai: Coming of age?," *Annals of Actuarial Science*, vol. 16, no. 1, pp. 1–5, 2022.

[2] J. H. Korteling, G. van de Boer-Visschedijk, R. A. Blankendaal, R. Boonekamp, and A. Eikelboom, "Human-versus artificial intelligence," *Frontiers in artificial intelligence*, vol. 4, p. 622364, 2021.

[3] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, *et al.*, "Highly accurate protein structure prediction with alphafold," *Nature*, vol. 596, no. 7873, pp. 583–589, 2021.

[4] H. Tann, S. Hashemi, R. I. Bahar, and S. Reda, "Hardware-software codesign of accurate, multiplier-free deep neural networks," in *2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*, pp. 1–6, IEEE, 2017.

[5] E. Park, D. Kim, and S. Yoo, "Energy-efficient neural network accelerator based on outlier-aware low-precision computation," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, pp. 688–698, IEEE, 2018.

[6] S. Jain, S. Venkataramani, V. Srinivasan, J. Choi, P. Chuang, and L. Chang, "Compensated-dnn: energy efficient low-precision deep neural networks by compensating quantization errors," in *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*, pp. 1–6, IEEE, 2018.

[7] C.-J. Wu, R. Raghavendra, U. Gupta, B. Acun, N. Ardalani, K. Maeng, G. Chang, F. Aga, J. Huang, C. Bai, *et al.*, "Sustainable ai: Environmental implications, challenges and opportunities," *Proceedings of Machine Learning and Systems*, vol. 4, pp. 795–813, 2022.

[8] G. Guzun and G. Canahuate, "Distributed query-aware quantization for high-dimensional similarity searches.," in *Advances in database technology: proceedings. International Conference on Extending Database Technology*, vol. 2018, pp. 373–384, 2018.

[9] J. Fowers, K. Ovtcharov, M. Papamichael, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman, L. Adams, M. Ghandi, S. Heil, P. Patel, A. Sapek, G. Weisz, L. Woods, S. Lanka, S. K. Reinhardt, A. M. Caulfield, E. S. Chung, and D. Burger, "A configurable cloud-scale dnn processor for real-time ai," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, pp. 1–14, 2018.

[10] A. Rodriguez, E. Segal, E. Meiri, E. Fomenko, Y. J. Kim, H. Shen, and B. Ziv, "Lower numerical precision deep learning inference and training," *Intel White Paper*, vol. 3, no. 1, p. 19, 2018.

[11] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *International conference on machine learning*, pp. 1737–1746, PMLR, 2015.

[12] N. Wang, J. Choi, D. Brand, C.-Y. Chen, and K. Gopalakrishnan, "Training deep neural networks with 8-bit floating point numbers," *Advances in neural information processing systems*, vol. 31, 2018.

[13] J. Lee, C. Kim, S. Kang, D. Shin, S. Kim, and H.-J. Yoo, "Unpu: An energy-efficient deep neural network accelerator with fully variable weight bit precision," *IEEE Journal of Solid-State Circuits*, vol. 54, no. 1, pp. 173–185, 2018.

[14] R. Mishra, H. P. Gupta, and T. Dutta, "A survey on deep neural network compression: Challenges, overview, and solutions," 2020.

[15] T. Dettmers, R. Svirschevski, V. Egiazarian, D. Kuznedelev, E. Frantar, S. Ashkboos, A. Borzunov, T. Hoefler, and D. Alistarh, "Spqr: A sparse-quantized representation for near-lossless llm weight compression," 2023.

[16] J. Yang, X. Shen, J. Xing, X. Tian, H. Li, B. Deng, J. Huang, and X.-s. Hua, "Quantization networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

[17] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "Eie: Efficient inference engine on compressed deep neural network," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 243–254, 2016.

[18] W. You and C. Wu, "Rsnn: A software/hardware co-optimized framework for sparse convolutional neural networks on fpgas," *IEEE Access*, vol. 9, pp. 949–960, 2021.

[19] Y. Liang, L. Lu, and J. Xie, "Omni: A framework for integrating hardware and software optimizations for sparse cnns," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 8, pp. 1648–1661, 2021.

[20] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.

[21] G. Guzun, J. Tosado, and G. Canahuate, "Slicing the dimensionality: Top-k query processing for high-dimensional spaces," in *Transactions on Large-Scale Data-and Knowledge-Centered Systems XIV*, pp. 26–50, Springer, 2014.